

Externe Daten

Aufgabe: Ein Array soll mit Werten aus einer Datei gefüllt und dann angezeigt werden.

Externe Dateien: CSV oder XML?

1. *Schritt: Geeigneten Dateityp auswählen*
   CSV - Daten in Form einer Tabelle, in jeder Zeile hat man gleichartige Daten
   - Werte durch Komma(,) oder Strichpunkt (;) getrennt (comma separated values)
   - Texte z.T. in Anführungszeichen (")
   - können eine Zeile mit Überschrift enthalten
   XML - Daten in Form eines Baums (extended markup language)
   - geeignet, wenn nicht für jeden Eintrag die gleichen Daten vorliegen.
2. *Schritt: Datei laden/speichern*
   CSV - Verwende die Klassen Table.
   XML - Verwende die Klasse XML.
3. *Schritt: Übertrage die Werte in ein Array.*
   In der Regel werden in einer `for`-Schleife alle Werte des Arrays einzeln belegt.
   z.B.
```
for(int i=0; i < kommazahlen.length; i++) {
    // i => i. Zeile, 0 => 1. Spalte
   kommazahlen[i] = csv.getFloat(i,0);
}
```

Aufgabe externe Datei:

Lade die Datei punkte.csv mit einem Texteditor (nicht mit einer Tabellenkalkulation!):

1. *Untersuche, wie viele Zeilen und wie viele Spalten sie hat. Ist es eine csv-Datei mit Überschriften? Welches Trennzeichen wurde zwischen den Werten verwendet? Wurden alle Texte in Anführungszeichen eingeschlossen?*
   Ergänze die Datei um zwei eigene Einträge (so kannst du cheaten...).

Datentypen

Nachdem man die Struktur der Daten kennt, kann man sich überlegen, welche Datentypen dazu passen:

Art der Daten	Datentyp	Befehl zum Auslesen aus CSV-Datei
Ganze Zahlen	int	csv.getInt(i,"Punkte");
Dezimalzahlen	double oder float	csv.getFloat(i, "Messwert");
Texte	String	csv.getString(i, "Name");

Punktschreibweise

Die „Tabelle“ `csv` vom Typ `Table` hat bestimmte Fähigkeiten. Zum Beispiel kann man Werte aus einer bestimmten Zelle auslesen, wie oben in der rechten Spalte dargestellt. Dabei gibt i die Zeile an und z.B. „Punkte“ die Überschrift der gewünschten Spalte.

Zunächst müssen wir im Programm aber die csv-Datei laden:

```
Table csv = loadTable("punkte.csv","header, csv");
```



Dabei ist punkte.csv der Name der Datei, in der die Daten liegen. „header, csv“ gibt an, dass die Tabelle eine Überschrift besitzt (header) und als csv-Datei abgespeichert wurde.

Aufgabe Implementation:

*Lade deine **erste** fertige Datei `balkendiagramm` und speichere sie unter `balkendiagramm_csv` ab. Kopiere dann die Datei `punkte.csv` in den gleichen Ordner, in dem auch deine Processing-Datei liegt.*

2. *Lösche wieder beim Array `zahlen` die initialisierten Werte. Deklariere ein weiteres Array `namen` für die Spielernamen (welcher Typ ist geeignet?).*
3. *Schreibe eine Methode `ladeTabelle(String name).`*
4. *Lade zunächst die Daten aus der csv-Datei:*
   `Table csv = loadTable(name,"header, csv");`
5. *Initialisiere dann die Arrays in der richtigen Größe. Die Länge der Arrays muss der Anzahl der Zeilen mit Werten deiner csv-Tabelle entsprechen. Diese kannst du mit `csv.getRowCount()` abfragen.*
6. *Fülle sie dann mit den Werten aus der Tabelle in einer Zählschleife. Nutze dazu die Methoden `getInt(i,"Punkte")` und `getString(i,"Name")` für das Auslesen der Werte in der i-ten Zeile.*
7. *Ändere die Methode `zeichneBalken()` so, dass immer ein Name und daneben der Balken mit dem passenden Punktwert dargestellt werden.*
8. *Rufe vor der Methode `zeichneBalken()` in der setup-Methode die Methode `ladeTabelle(„punkt.csv“)` auf und teste dein Programm.*

Testen der Implementation:

Da hier Daten aus einer externen Datei kommen, können hier viele Fehler passieren. Zum Beispiel könnte die Datei gar nicht existieren oder keine Überschriften enthalten.

9. *Überlege dir noch zwei Fehler, die die Daten haben könnten.*
10. *Kopiere die Datei punkte.csv, passe das Programm so an, dass deine Kopie geladen wird, und experimentiere, was bei den Fehlern passiert.*
11. *Lies die Dokumentation zum Befehl, der zum Laden der Datei verwendet wird, und versuche das Programm so anzupassen, dass es mit einer Datei ohne Überschriften zurecht kommt. (https://processing.org/reference/loadTable_.html)*
12. *Versuche möglichst viele Fehler abzufangen (z.B. das Fehlen der Datei).*